Exception Handling

12 June 2024 09:56

Exception handling in Java is a powerful mechanism that allows developers to manage runtime errors and maintain a normal slow of application execution. It enables the handling of exceptional condition in enough. controlled way, preventing the program from terminating abruptly.

Key concepts of Exception Handling

1. **Exception:** An exception is an event that disrupts the normal flow of the program's instruction during runtime.
2. **Exception Handling:** The process of responding to the occurrence of exceptions.

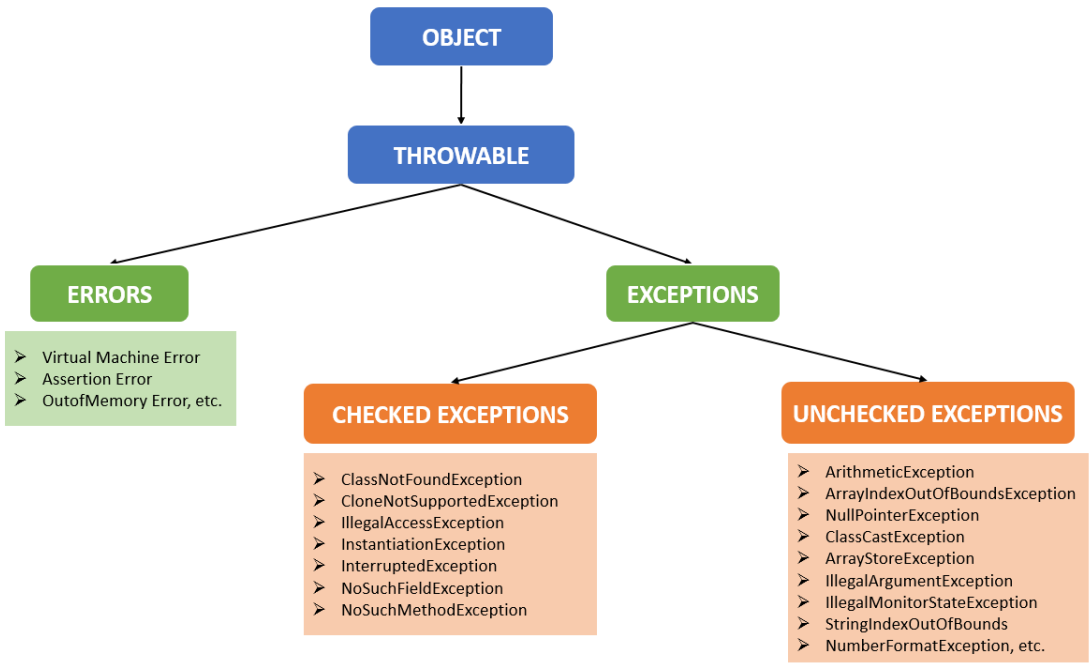
Types of Exceptions

1. **Checked Exception:** These exceptions are checked at compile time. Examples include 'IOException', 'SQLException', and 'ClassNotFoundException'.
2. **Unchecked Exceptions:** These exceptions are checked at runtime. They are also known as runtime exceptions. Examples include. 'ArithmeticException', 'NullPointerException', 'ArrayIndexOutOfBoundsException'.
3. **Error:** Errors are serious issues, that a reasonable application should not try to catch examples include 'OutOfMemoryError' and 'StackOverflowError'.

Exception Hierarchy

In Java, all exceptions are the part of the **'Throwable'** class. The hierarchy is as follows:

- **Throwable**
  - **Exception**
    - **RuntimeException (Unchecked Exception)**
    - **(other checked exceptions like 'IOException', 'SQLException')**
  - **Error (not meant to be caught or handled by programs)**



Basic Syntax for Exception Handling

Java provides five keywords for exception handling.

- 'try'
- 'catch'
- 'finally'
- 'throw'
- 'throws'

'try-catch' block

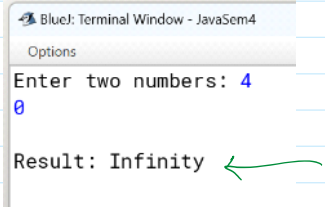
The **'try'** block contains code that might throw an exception. The **'catch'** block contains code to handle the exception.

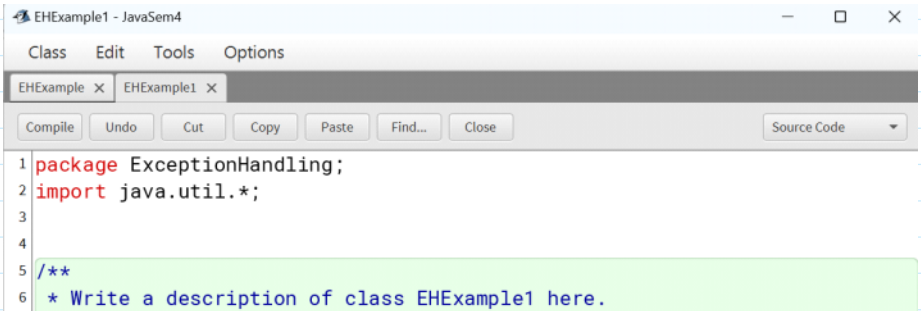
```
try{
    //code that may throw an exception
} catch(ExceptionType e){
    //Code to handle the exception
}
```

```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class EHExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample
{
   public static void main(String args[]){
      double x, y;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter two numbers: ");
      x = sc.nextDouble();
      y = sc.nextDouble();
      double r = x / y;
      System.out.print("\nResult: " + r);
   }
}
```

Output:





EHExample1 - JavaSem4

Class Edit Tools Options

EHExample | EHExample1

Compile Undo Cut Copy Paste Find... Close Source Code

```java
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class EHExample1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample1
{
    public static void main(String args[]){
        int x, y;
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter two numbers: ");
        x = sc.nextInt();
        y = sc.nextInt();
        int r = x / y;
        System.out.print("\nResult: " + r);
    }
}
```

java.lang.ArithmeticException:
/ by zero

saved

In case of integer you have to handle ArithmeticException.

BlueJ: Terminal Window - JavaSem4

Options

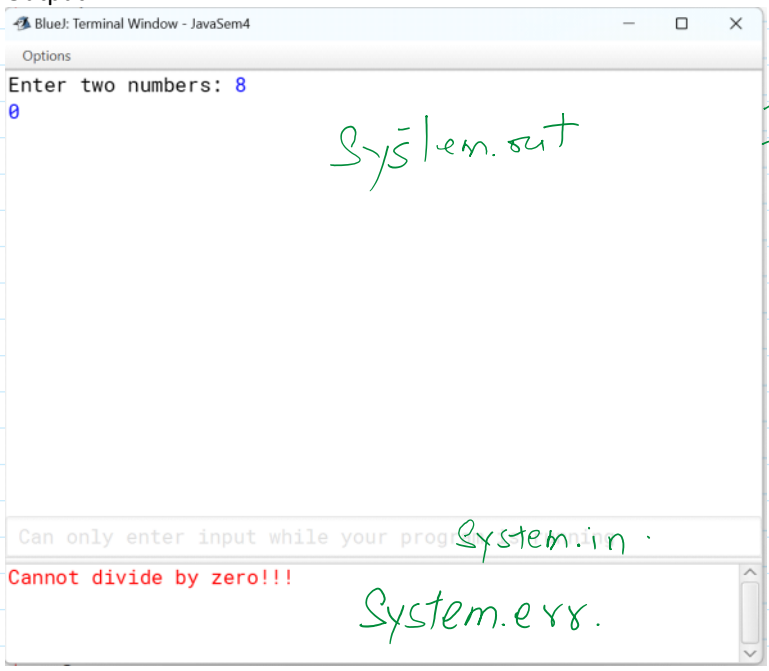
```
Enter two numbers: 9
0
```

```java
package ExceptionHandling;
import java.util.*;

/**
 * Write a description of class EHExample1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample1
{
   public static void main(String args[]){
      int x, y, r = 0;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter two numbers: ");
      x = sc.nextInt();
      y = sc.nextInt();
      try{
         r = x / y;
      }
      catch(ArithmeticException e){
         System.err.print("\nCannot divide by zero!!!");
         return; //exiting the main code.
      }
      System.out.print("\nResult: " + r);
```

```
    }
  }
```

Output:



Input Streams of Java
System.out
System.in
System.err.

Finally Block

The **'finally'** block contains code that is always executed, regardless of whether an exception is thrown or not.

Syntax:

```
    try{
        // Code that may throw an exception
    }
   catch(Exception e){
        // Code to handle exception
   }
   finally {
        //Code that will always execute
   }
```

For example:

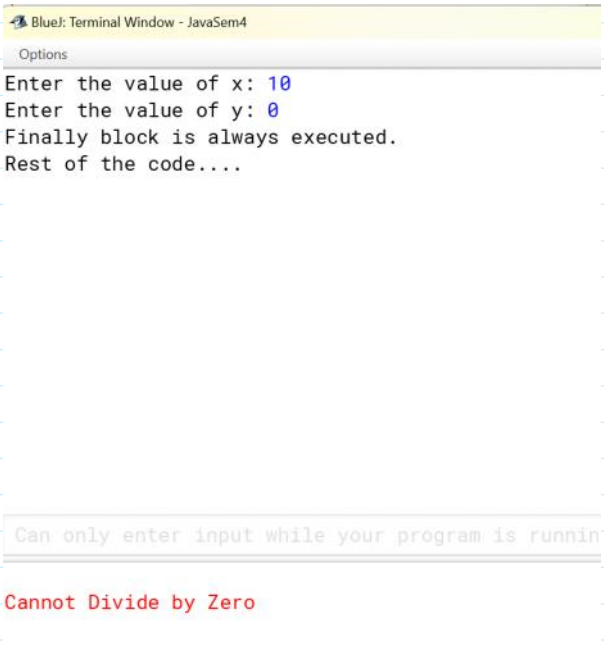
```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class EHExample2 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample2
{
  public static void main(String args[]){
    Scanner sc = new Scanner(System.in);
```

```
    int x, y, z = 0;
    System.out.print("\fEnter the value of x: ");
    x = sc.nextInt();
    System.out.print("Enter the value of y: ");
    y = sc.nextInt();

    try{
      z = x / y;
      System.out.print("\nResult: " + z);
    }
    catch(ArithmeticException ae){
      System.err.print("\nCannot Divide by Zero");
    }
    finally{
      System.out.print("Finally block is always executed.");
    }
    System.out.print("\nRest of the code....");
  }
}
```

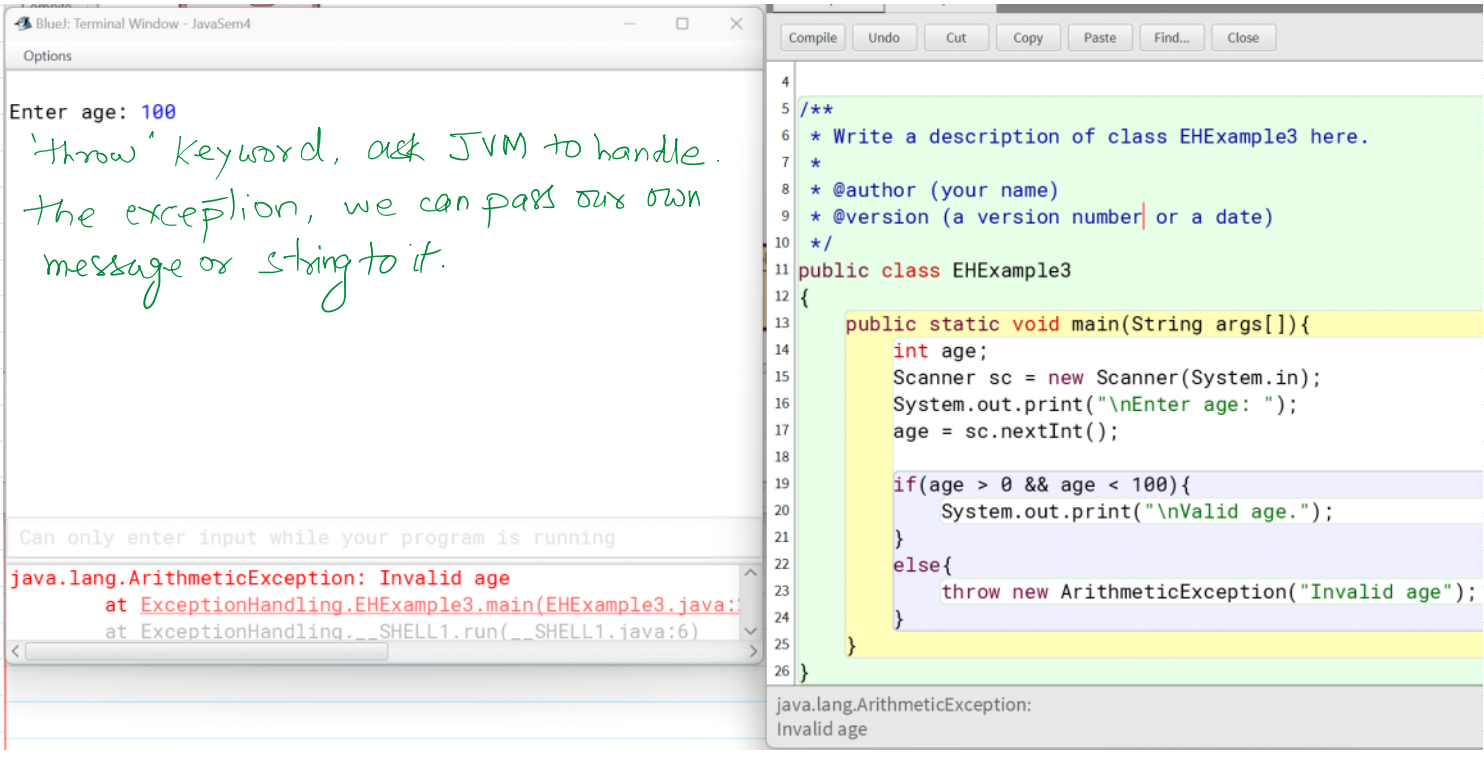
Output:



'throw' keyword

The **'throw' keyword** is used to explicitly throw an exception.

For example:



'throws' Keyword

The **'throws' Keyword** is used to declare an exception in the method signature, indicating that method might throw this exception.

For example:

```
package ExceptionHandling;
import java.io.*;


/**
 * Write a description of class EHExample4 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample4
{
   public static void main(String args[]){
      try{
         method();
      }catch(IOException e){
         System.out.print("\nIOException handled");
      }
   }

   public static void method() throws IOException
   {
      throw new IOException("Device Error");
   }
}
```

← method signature.

Another example:

```
package ExceptionHandling;
import java.io.*;
```

```
/**
 * Write a description of class EHExample5 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample5
{
   public static void anyFun() throws IOException, ArithmeticException {
      InputStreamReader isr = new InputStreamReader(System.in);
      BufferedReader br = new BufferedReader(isr);
      int x;
      float f;
      String wrd;
      System.out.print("\fEnter the value of x: ");
      x = Integer.parseInt(br.readLine());
      System.out.print("Enter the value of f: ");
      f = Float.parseFloat(br.readLine());
      System.out.print("\nEnter any word: ");
      wrd = br.readLine();
      System.out.print("\nx : " + x);
      System.out.print("\nf : " + f);
      System.out.print("\nwrd : " + wrd);
      System.out.print("\nresult: " + (x/0));
   }

   public static void main(String args[]){

      try
      {
         anyFun();
      }
      catch(ArithmeticException ae){
         System.err.print("Cannot divide by 0\n");
      }
      catch(NumberFormatException nfe){
         System.err.print("\nDifferent symbol other than digits is entered");
      }
      catch (IOException ioe)
      {
         System.err.print("Data input error!");
      }
   }
}
```

keyword mostly used with checked exception.

Checked Exception.

Unchecked Exception

Nested try Block

Example of Nested try Block

```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class EHExample6 here.
 *
```

```
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample6
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      int x, y, r;
      System.out.print("\fEnter any integer value: ");
      x = sc.nextInt();
      System.out.print("Enter any integer value: ");
      y = sc.nextInt();
      int ar[] = new int[5];
      try{
         try{
            r = x / y;
            System.out.print("\nResult = " + r);
         }
         catch(ArithmeticException e){
            System.err.print("\nCannot divide by zero.");
         }
         try{
            System.out.print("\nEnter 5 integer values: \n");
            for(int i = 0; i <= ar.length; i++){
               ar[i] = sc.nextInt();
            }
         }
         catch(ArrayIndexOutOfBoundsException aiobe){
            System.err.print("Array index out of bounds.");
         }
      }
      catch(Exception e){
         System.err.print("\nHandle other exceptions");
      }
      System.out.print("\nRest of the code...");
   }
}
```

Execute the code and check different types of output.

Exception Propagation

Exception propagation occurs when an exception is thrown from a method and then not caught within that method. The exception propagates to the caller method.

For example:

```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class EHExample7 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class EHExample7
{
```

```
  public static void main(String args[]){
    int x, y;
    Scanner sc = new Scanner(System.in);
    try{
      System.out.print("\fEnter the value of x: ");
      x = sc.nextInt();
      System.out.print("Enter the value of y: ");
      y = sc.nextInt();
21    method1(x, y);
    }
    catch(Exception e){
      System.err.print("\nException is handled in main()");
      System.err.print("\n" + e.getMessage()+"\n\n");
      e.printStackTrace();
    }
  }

  public static void method1(int x, int y){
31  method2(x, y);
  }

  public static void method2(int x, int y){
35  method3(x, y);
  }

  public static void method3(int x, int y){
39  int r = x/y;
    System.out.print("\nResult = " + r);
  }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
Enter the value of x: 10
Enter the value of y: 0

Can only enter input while your program is running

Exception is handled in main()
/ by zero

java.lang.ArithmeticException: / by zero
        at ExceptionHandling.EHExample7.method3(EHExample7.java:39)
        at ExceptionHandling.EHExample7.method2(EHExample7.java:35)
        at ExceptionHandling.EHExample7.method1(EHExample7.java:31)
        at ExceptionHandling.EHExample7.main(EHExample7.java:21)
        at ExceptionHandling.__SHELL7.run(__SHELL7.java:6)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at bluej.runtime.ExecServer$3.lambda$run$0(ExecServer.java:873)
        at bluej.runtime.ExecServer.runOnTargetThread(ExecServer.java:996)
        at bluej.runtime.ExecServer$3.run(ExecServer.java:870)
```

Best Practices for exception handling

1. **Catch specific exceptions:** Catch specific exception rather than a generic **'Exception'** to handle different scenarios properly.
2. **Use Finally Block:** Ensure resources. are closed properly by using **'finally'** block or with try-with-resource statement.
3. **Custom Exception Messages:** Provides meaningful exception messages to make debugging easier.
4. **Avoid Silent Failures:** Do not catch exception without handling them or logging them.
5. **Document Exception:** Use the **'@throws'** Javadoc tag to document the exceptions a method might throw.

By following these principles and techniques, you can effectively manage exceptions in Java, making your code more robust and maintainable.